

$F2 \parallel wC_{\max}$ Structural Properties and Dynamic Programming

Author One^{a,*}, Author Two^b

^aInstitution One, Location, Country

^bInstitution Two, Location, Country

Abstract

Minimizing $wC_{\max}$ on a two-machine flow shop, denoted $F2 \parallel wC_{\max}$, is a classical NP-hard scheduling problem. Existing exact methods rely on enumerative techniques that scale poorly with instance size, while approximation algorithms often impose restrictive assumptions on processing times. We first sequence all jobs globally by Johnson's rule, obtaining an initial schedule π . Scanning π from left to right, we partition it into at most K blocks: whenever a weight not seen before appears, it serves as the final job of the current block, and the next job starts a new block. We establish structural properties of optimal schedules: the permutation property, the positional constraint that block $\mathcal{G}_k$ jobs cannot appear beyond the first k blocks, and the block-end condition requiring the final job of each block to carry that block's weight (automatically satisfied by construction). The intra-block Johnson ordering inherited from π is further formalized within the DP framework. Building on these properties and the interval-based dynamic programming framework (Hall, 1998) developed for $F2|r_j|C_{\max}$, we design a polynomial-time dynamic programming algorithm with $O(|Y|_{\max} \cdot n)$ complexity that appends jobs block by block. We further show that geometric rounding of weights and processing times converts this algorithm into a polynomial-time approximation scheme.

Keywords: Flow shop scheduling; $wC_{\max}$; Dynamic programming; Structural properties

1 Introduction

2 Preliminaries

We consider the problem $F2 \parallel wC_{\max}$. Let $\mathcal{J} = \{J_1, J_2, \dots, J_n\}$ be the set of n jobs to be processed on two machines M_1 and M_2 in series: each job J_j must be processed first on

*Corresponding author. E-mail address: author@email.com

M_1 for a_j time units and then on M_2 for b_j time units, with no preemption allowed. M_1 and M_2 can process at most one job at a time. Each job J_j has a weight $w_j > 0$. A schedule $\pi = (\pi(1), \pi(2), \dots, \pi(n))$ specifies the processing order on both machines. For a schedule π , let $A_j = \sum_{h=1}^j a_{\pi(h)}$ and $B_j = \sum_{h=1}^j b_{\pi(h)}$ be the cumulative M_1 and M_2 completion times after the first j jobs.

The objective is to find a schedule π that minimizes $wC_{\max}$.

For $F2 \parallel wC_{\max}$, a permutation schedule in which the job order on M_1 and M_2 , is optimal (Baker, 1974). Johnson's rule (Johnson, 1954) sequences all n jobs to minimize the makespan,

Lemma 2.1. *For $F2 \parallel wC_{\max}$, there exists an optimal schedule in which all jobs are sequenced according to Johnson's rule.*

Proof. □

3 NP-hard

Theorem 3.1. *$F2 \parallel wC_{\max}$ is NP-hard.*

Proof. We reduce from the Partition problem: given n positive integers $x_1, x_2, \dots, x_r$ with $\sum_{i=1}^r x_i = 2T$, does there exist subset Z_1 and $Z_2 \subseteq \{1, \dots, r\}$ such that: $\sum_{i \in Z_1} x_i = \sum_{i \in Z_2} x_i = T$?

Construct an instance of $F2 \parallel wC_{\max}$, Let the number of jobs $n = r + 1$. Let $a_j = 1$, $b_j = rx_j$, $w_j = T + 1$ for jobs $j = 1, \dots, n - 1$. Put $a_n = rT$, $b_n = 1$, $w_n = 2T + 1$ for job J_n . Does there exist a threshold Y such that:

$$wC_{\max} \leq Y = r(T + 1)(2T + 1)$$

holds?

We partition jobs j_1 to j_{n-1} into two subsets via j_n , the distribution of jobs on the machine is $(\mathcal{J}_{j \in A_1}, J_n, \mathcal{J}_{j \in A_2})$. Since the processing times of A_1 and A_2 are unknown, while their total processing time is $2T$. We may assume that the processing time of A_1 on M_2 is $r(T + \xi)$, A_2 is $r(T - \xi)$, where $-T \leq \xi \leq T$. Therefore, the completion time of J_n is determined by the following formula:

$$C_n^1 = |A_1| + rT + 1 \tag{1}$$

$$C_n^2 = 1 + r(T + \xi) + 1 \tag{2}$$

and $C_n = \max\{C_n^1, C_n^2\}$

If $\xi = 0$, then $\sum_{j \in A_1} b_j = \sum_{j \in A_2} b_j$, which means the partition problem has a feasible solution. In this case, the completion time of job J_n is determined by (1), we have:

$$\begin{aligned}
w_n C_n &= (2T + 1)(|A_1| + rT + 1) \\
&\leq (2T + 1)(rT + r) \\
&= r(T + 1)(2T + 1) \\
&= Y \\
w C_{\max} &= r(T + 1)(C_n + rT) = Y
\end{aligned}$$

so $w_j C_j \leq w C_{\max} = Y$

If the scheduling problem exists a threshold y such that $w C_{\max} \leq y$. When $\xi > 0$, C_n is determined by (2), we have:

$$\begin{aligned}
w_n C_n &= (1 + r(T + \xi) + 1)(2T + 1) \\
&\geq (1 + r(T + 1) + 1)(2T + 1) \\
&> Y
\end{aligned}$$

When $\xi < 0$, C_n is determined by (1), we have:

$$\begin{aligned}
w C_{\max} &= (C_n + \sum_{j \in A_2} b_j)(T + 1) \\
&= (1 + rT + |A_1| + r(T - \xi))(T + 1) \\
&\geq (rT + r + rT)(T + 1) \\
&= r(T + 1)(2T + 1) = Y
\end{aligned}$$

so $w C_{\max} > Y$, there exists a contradiction.

Therefore, the Partition instance admits a solution if and only if the constructed $F2 \parallel w C_{\max}$ instance admits a schedule with $w C_{\max} \leq Y$.

□

4 $F2 \parallel w C_{\max}$ for the number of weights is constant

When the number of distinct weights K is bounded by a constant, the structural properties established in Section 2 enable a polynomial-time dynamic programming algorithm. By Lemma 2.1, the initial job sequence follows Johnson's rule; the block partition and all subsequent steps are defined with respect to this Johnson order. We first describe the block partition and group logic underlying our approach, then present the DP formulation, and finally give the complete algorithm and its complexity analysis.

4.1 Algorithm Properties and Dynamic Programming

The jobs are partitioned into blocks by a single left-to-right scan. Starting from the first job, a new block is created whenever a weight value not encountered before appears; that job becomes the final job of the current block, and the next job starts a new block. Since each block introduces at least one previously unseen weight, the procedure yields at most K blocks, denoted $\mathcal{B}_1, \mathcal{B}_2, \dots, \mathcal{B}_K$. By construction, the final job of each block carries the weight that triggered the cut. By construction, block weights are non-increasing, i.e., $W(\mathcal{B}_1) \geq W(\mathcal{B}_2) \geq \dots \geq W(\mathcal{B}_K)$.

Jobs of the same weight form a natural group, denoted $\mathcal{G}_1, \mathcal{G}_2, \dots, \mathcal{G}_K$. Scheduling heavier groups before lighter ones automatically enforces the non-increasing block-weight condition, and scanning groups in descending weight order directly yields the block partition.

Example 4.1. Consider a 6-job instance with Johnson sequence $\pi = (J_1, \dots, J_6)$ and weights $(w_1, \dots, w_6) = (2, 2, 5, 2, 4, 2)$. Scanning π from left to right and cutting whenever a weight not seen before appears yields the following partition:

$$\underbrace{J_1, J_2, J_3}_{\mathcal{B}_1} \quad \underbrace{J_4, J_5}_{\mathcal{B}_2} \quad \underbrace{J_6}_{\mathcal{B}_3}$$

The resulting blocks are $\mathcal{B}_1 = \{J_1, J_2, J_3\}$, $\mathcal{B}_2 = \{J_4, J_5\}$, $\mathcal{B}_3 = \{J_6\}$. And $\mathcal{G}_1 = \{J_1, J_2, J_4, J_6\}$, $\mathcal{G}_2 = \{J_5\}$, $\mathcal{G}_3 = \{J_3\}$.

Lemma 4.1. For a given schedule π , rescheduling each blocks for $\mathcal{B}_1, \mathcal{B}_2, \dots, \mathcal{B}_K$ to Johnson's algorithm and concatenating blocks schedules delivers a schedule of length at most $C(\pi)$

Proof. □

We now develop a dynamic programming algorithm that processes jobs sequentially in Johnson order and assigns each job to one of the K blocks $\mathcal{B}_1, \dots, \mathcal{B}_K$. Lemma 4.1 provides the structural basis: it guarantees that, given any assignment of jobs to the K blocks, sequencing the jobs within each block $\mathcal{B}_i$ according to Johnson's rule and concatenating the blocks in non-increasing weight order yields a schedule whose makespan is no larger than that of any other schedule respecting the same assignment. We denote by $\mathcal{H}$ the set of states maintained during the computation; each state summarizes the aggregate parameters of all K blocks after a prefix of jobs has been processed.

State Definition. A state is a $4K$ -tuple

$$(C_1, \dots, C_K; A_1, \dots, A_K; B_1, \dots, B_K; L_1, \dots, L_K) \in \mathcal{H}$$

where for each block $\mathcal{B}_i$, $i = 1, \dots, K$:

- A_i — total processing time on machine M_1 of jobs assigned to $\mathcal{B}_i$;

- B_i — total processing time on machine M_2 of jobs assigned to $\mathcal{B}_i$;
- C_i — makespan of $\mathcal{B}_i$ when the jobs assigned to it are scheduled in isolation;
- L_i — weight of the last job in $\mathcal{B}_i$ (or 0 if $\mathcal{B}_i$ is empty).

The initial state set contains a single state with all components set to zero: $\mathcal{H}^{(0)} = \{(0, \dots, 0)\}$.

Lemma 4.2. *Let S and S' be two states in $\mathcal{H}$ after processing the same prefix of jobs. If $A_i \leq A'_i$, $B_i \leq B'_i$, $C_i \leq C'_i$, and $L_i \leq L'_i$ for all $i = 1, \dots, K$, then S' is dominated by S and can be removed from further consideration.*

Proof. Let the remaining jobs be processed identically from both states. By the monotonicity of the recurrence in (5) and the concatenation formula, smaller A_i , B_i , C_i values lead to no larger final makespan. Smaller L_i ensures the weighted completion time objective is not increased. Hence S yields a schedule no worse than S' under $wC_{\max}$. $\square$

State Transition. For $j = 1, \dots, n$, we generate $\mathcal{H}^{(k)}$ from $\mathcal{H}^{(k-1)}$ as follows. Set $\mathcal{B}_i = \emptyset$ for $i = 1, \dots, K$. For each state $(C_1, \dots, C_K; A_1, \dots, A_K; B_1, \dots, B_K; L_1, \dots, L_K) \in \mathcal{H}^{(k-1)}$ and each block index $i = 1, \dots, K$. Job J_j may be assigned to the first i blocks whose index does not exceed the rank of its weight, then add a new state to $\mathcal{B}_i$ with the i -th block updated as follows:

$$A'_i = A_i + a_j, \tag{3}$$

$$B'_i = B_i + b_j, \tag{4}$$

$$C'_i = \max\{C_i + b_j, A_i + a_j + b_j\}, \tag{5}$$

$$L'_i = w_j. \tag{6}$$

All other components remain unchanged. Merge $\mathcal{B}_1, \dots, \mathcal{B}_K$ and apply the dominance rule (Lemma 4.2) to obtain $\mathcal{H}^{(k)}$.

Once all jobs are assigned and each block $\mathcal{B}_i$ is summarized by its parameters (A_i, B_i, C_i) , the overall makespan $C_{\max}$ is obtained by concatenating the blocks in order of increasing index. The procedure is given below:

In the update of T_B , the term $T_A - A_i$ is the start time of $\mathcal{B}_i$ on M_1 , so $T_A - A_i + C_i$ is the completion time of $\mathcal{B}_i$ on M_2 when M_2 is free, while $T_B + B_i$ accounts for the case where M_2 is the bottleneck.

Algorithm 1

Require: Block parameters (A_i, B_i, C_i) for $i = 1, \dots, K$.

Ensure: Completion times (T_A, T_B) after concatenating all K blocks, where $T_B = C_{\max}$.

```
1:  $T_A \leftarrow 0, \quad T_B \leftarrow 0$ 
2: for  $i = 1$  to  $K$  do
3:    $T_A \leftarrow T_A + A_i$ 
4:    $T_B \leftarrow \max\{T_A - A_i + C_i, T_B + B_i\}$ 
5: end for
6: return  $(T_A, T_B)$ 
```

Algorithm 2

Require: Jobs $\mathcal{J}$ with (a_j, b_j, w_j) , numbered $1, \dots, n$ in Johnson order.

Ensure: The optimal makespan $C_{\max}$.

```
1:  $\mathcal{H} \leftarrow \{(0, \dots, 0)\}$ 
2: for  $j = 1$  to  $n$  do
3:    $\mathcal{Y}_i \leftarrow \emptyset$  for  $i = 1, \dots, K$ 
4:   for each  $(C_1, \dots, C_K; A_1, \dots, A_K; B_1, \dots, B_K; L_1, \dots, L_K) \in \mathcal{H}$  do
5:     for  $i = 1$  to  $\ell(j)$  do
6:        $A'_i \leftarrow A_i + a_j$ 
7:        $B'_i \leftarrow B_i + b_j$ 
8:        $C'_i \leftarrow \max\{C_i + b_j, A_i + a_j + b_j\}$ 
9:        $L'_i \leftarrow w_j$ 
10:      Add  $(\dots, C'_i, \dots; \dots, A'_i, \dots; \dots, B'_i, \dots; \dots, L'_i, \dots)$  to  $\mathcal{Y}_i$ 
11:    end for
12:  end for
13:   $\mathcal{H} \leftarrow \text{MERGE}(\mathcal{Y}_1, \dots, \mathcal{Y}_K)$ 
14: end for
15:  $C_{\max} \leftarrow +\infty$ 
16: for each state  $S \in \mathcal{H}$  do
17:    $(T_A, T_B) \leftarrow \text{Algorithm 1}((A_1, B_1, C_1), \dots, (A_K, B_K, C_K))$ 
18:    $C_{\max} \leftarrow \min\{C_{\max}, T_B\}$ 
19: end for
20: return  $C_{\max}$ 
```

The algorithm above computes the optimal makespan $C_{\max}$. To obtain the weighted objective $wC_{\max}$ from a state $S \in \mathcal{H}$, let $T_A^{(i)}$ and $T_B^{(i)}$ be the completion times on M_1 and M_2 after concatenating blocks $\mathcal{B}_1$ through $\mathcal{B}_i$ via the Algorithmic 1. Since blocks are concatenated in non-increasing weight order, $T_B^{(i)}$ is precisely the completion time of the last

job in $\mathcal{B}_i$. The weighted completion time of the schedule corresponding to S is therefore

$$wC_{\max}(S) = \max_{i=1,\dots,K} L_i \cdot T_B^{(i)},$$

where L_i is the weight recorded for $\mathcal{B}_i$ (the weight of its last job). Taking the minimum over all states in $\mathcal{H}$ yields the optimal objective value:

$$wC_{\max}^* = \min_{S \in \mathcal{H}} \max_{i=1,\dots,K} L_i \cdot T_B^{(i)}.$$

Equivalently, this computation can be folded into Step 3 of Algorithm $\mathcal{H}$ by updating $\text{obj} \leftarrow \max\{\text{obj}, L_i \cdot T_B\}$ after each Algorithmic 1 iteration and returning $\min_S \text{obj}$ instead of $C_{\max}$.

Theorem 4.1. *Algorithm 1 computes an schedule for $F2 \parallel wC_{\max}$ in $O()$ time.*

Proof.

□

4.2 FPTAS

5 Conclusion

References

- Baker, K. R. (1974). *Introduction to Sequencing and Scheduling*. Wiley, New York.
- Hall, L. A. (1998). Approximability of flow shop scheduling. *Mathematical Programming*, 82(1–2):175–190.
- Johnson, S. M. (1954). Optimal two- and three-stage production schedules with setup times included. *Naval Research Logistics Quarterly*, 1(1):61–68.